

Design Decision Document

1. Primary Sensor Selection — DHT22 to BME280

What we were trying to achieve

A reliable environmental sensor capable of measuring temperature, humidity, and atmospheric pressure.

Options considered

- DHT22
- BME280

What we chose

BME280

Why

The DHT22 was initially used during early proof-of-concept testing because it was already available and allowed rapid validation of the environmental sensing workflow.

Once replacement BME280 units were obtained and correctly wired, the BME280 became the clear choice because it combines temperature, humidity, and barometric pressure sensing into a single device.

The BME280 better supported the weather prediction goals because pressure trends are important for classifying changing weather conditions.

2. Microcontroller Selection — ATmega328P-A

What we were trying to achieve

A capable embedded microcontroller suitable for a manufacturable custom PCB design.

Options considered

- Raspberry Pi
- Arduino Uno
- ATmega328P-A

What we chose

ATmega328P-A

Why

Early proof-of-concept testing used a Raspberry Pi platform, but it became clear during design review that a Raspberry Pi and touchscreen-based system would not reflect a practical manufacturable embedded product.

Using the ATmega328P-A directly on the PCB reduced unnecessary overhead, lowered system complexity, and better reflected professional embedded system design practices.

3. Communication to Computer — USB Serial

What we were trying to achieve

Reliable communication between the PCB and external software for environmental data transfer and analysis.

Options considered

- GPIO-only monitoring
- Wireless-only communication
- USB serial communication

What we chose

USB serial communication

Why

USB communication provided a reliable and straightforward method for transferring environmental data between the PCB and the external computer application.

This simplified debugging, testing, and software integration while allowing environmental data to be visualized and analyzed using Python-based software.

4. Power Input & Communication Interface — Dual USB-C Design

What we were trying to achieve

A cleaner and more modern solution for both powering the board and transferring environmental data between the PCB and external software.

Options considered

- Screw terminals
- Barrel jack
- Single USB-C connector
- Dual USB-C connectors

What we chose

Two dedicated USB-C connectors:

- One USB-C connector for board power input
- One USB-C connector for communication and USB serial data transfer

Why

Earlier versions of the project relied on screw terminals for power input, but this approach felt less practical and less user-friendly for a modern embedded product.

Moving to USB-C simplified external connectivity and created a cleaner interface for both development and future expansion.

Separating power and communication into two dedicated USB-C connectors also simplified debugging and system organization.

5. Crystal Oscillator vs Internal Oscillator

What we were trying to achieve

Stable and accurate MCU timing.

Options considered

- Internal oscillator
- External crystal oscillator

What we chose

External crystal oscillator with 22pF load capacitors

Why

The external crystal oscillator provides more stable and accurate timing than the internal oscillator, improving communication reliability and overall system stability.

6. PCB Layer Count — 2 Layer vs 4 Layer

What we were trying to achieve

A clean and manufacturable PCB layout with improved routing flexibility.

Options considered

- 2-layer PCB
- 4-layer PCB

What we chose

Primary 2-layer design with experimentation using a 4-layer stackup.

Why

Initial routing on a standard 2-layer PCB became congested during layout development.

Experimenting with a 4-layer PCB improved understanding of professional PCB routing workflows and allowed exploration of inner-layer signal routing and via assignment techniques.

7. User Interface Approach

What we were trying to achieve

A practical method for users to interact with and monitor environmental data.

Options considered

- Touchscreen GUI
- Onboard display
- External software interface
- Mobile notifications

What we chose

USB communication with external software and future notification support.

Why

A touchscreen-based Raspberry Pi system introduced unnecessary complexity and cost.

Moving monitoring and visualization to external software simplified the PCB design while improving expandability for future features such as notifications and weather prediction.

8. Support Components

What we were trying to achieve

Stable operation and proper support circuitry for the embedded system.

What we chose

- 10k Ω reset pull-up resistor
- 4.7k Ω pull-up resistors
- 330 Ω LED resistor
- 22pF crystal load capacitors
- Decoupling capacitors
- Voltage regulator
- Test points
- ISP programming header

Why

Values and support components were selected based on datasheet recommendations and common embedded system design practices to ensure stable operation, clean power delivery, and reliable communication.

Several components were added or corrected during PCB review which improved the final design.